Packets, Frames and Error Detection

Chaminade University

Department of Computer Science and Computer Information Systems

Goals

- In this chapter you will learn about:
  - The concept of packets
  - How the sender and receiver coordinate to transfer a packet
  - How packets can be implemented
  - Transmission errors
  - Error detection

The Concept of Packets

- Data is not sent as an arbitrary string of continuous bits.
- Data is divided into small blocks called packets.
- Hence the term *packet networks* or *packet switching networks*.

Motivation for packets

- Two reasons to use packets:
  - Easier to implement error detection and recovery.
  - Help ensure fairness (by avoiding a computer to hold a network for an arbitrary time).

Example

An illustration of one reason computer networks use packets.
While one pair of computers communicate, others must wait.

Summarize

- To allow sender and receiver to coordinate and to ensure all computers that share a network resource have fair, prompt access, most computer networks divide data into *small blocks* called *packets.*
- Computers take turns sending packets over the shared resource.
- Because each packet is small, no computer experiences a long delay while waiting for access

Packets and TDM

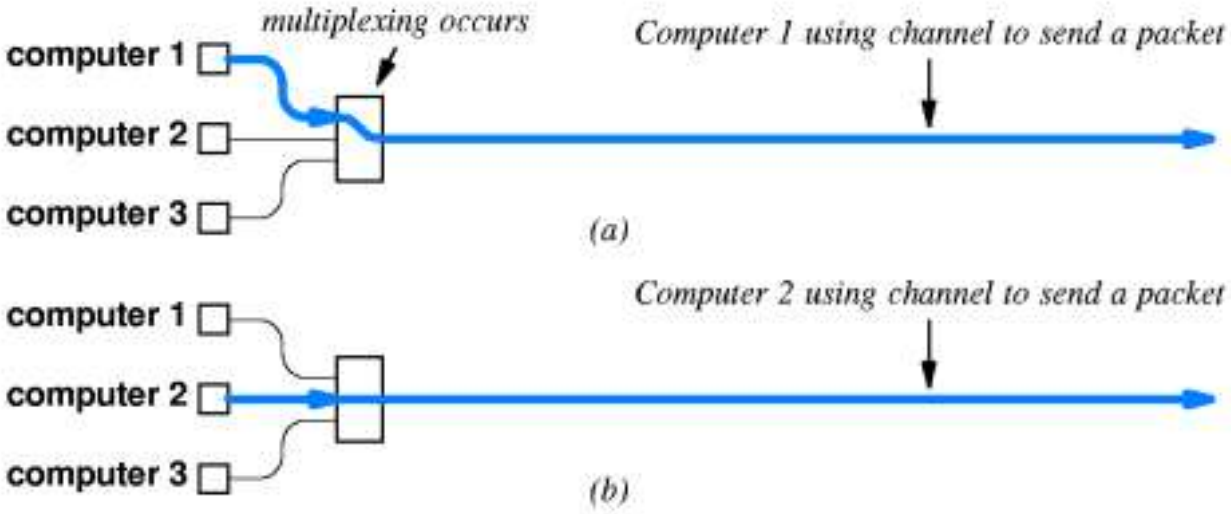


Illustration of multiplexing with packets. The source take turns using the shared communication channel. Computer 1 uses the resource to send a packet, and then computer 2 uses the resource to send a packet.

Packets and Hardware Frames

soh	block of data in frame	eot

An example frame that uses a character SOH (start of header) and EOT (end of transmission) to mark the beginning of the frame and EOT to mark the end.

ASCII includes these two characters primarily for framing. SOH = 01H and EOT = 04H

Packets and Frames

- Packets: the general idea of small block of data
- Frames: the definition of packets for a given hardware technology.

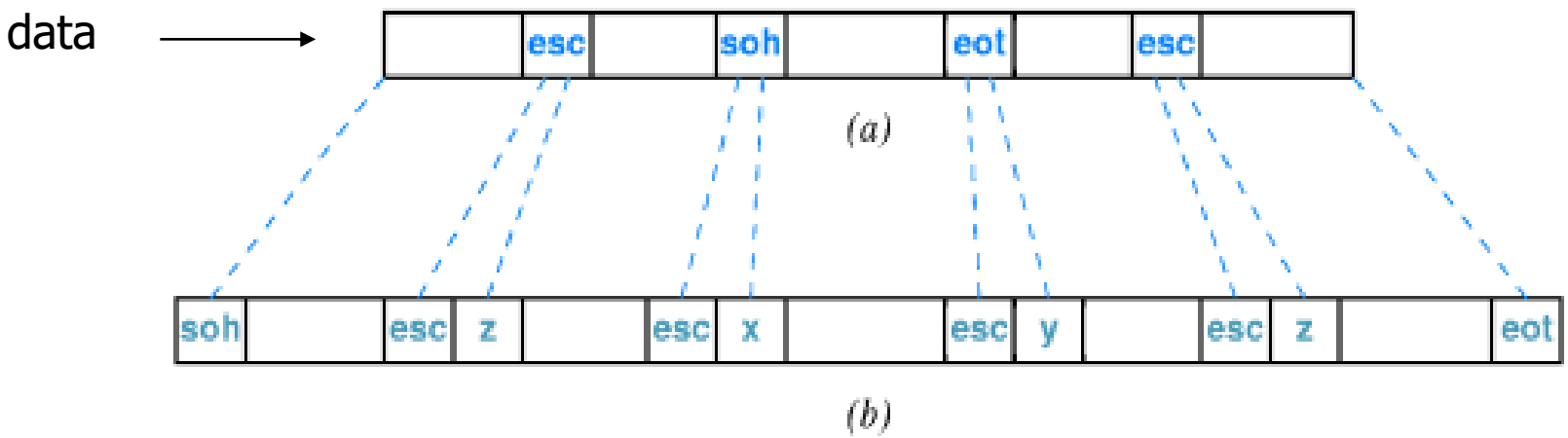
Byte Stuffing

- Most computer networks cannot afford to reserve characters for use by the network as frame delimiters.
- An application may choose to send data that contains one ore more occurrences of characters like SOH and EOT that are used for framing.
- A receiver will interpret EOT to mean that the entire frame has arrived.
- To avoid confusion of data with control information, the network insert extra bits or bytes, what is known as *data stuffing*.

Byte Stuffing

Character In Data	Characters Sent
soh	esc x
eot	esc y
esc	esc z

Byte Stuffing



a) Is an example of data that includes SOH and EOT b) Is the frame after byte stuffing. The dashed lines show the locations in the original data where characters have been replaced or new characters added.

Transmission Errors

- Sources of Interference
  - Lightning
  - Power-surges
  - Electro-magnetic Interference
- Interference over
  - Busy circuits – corrupt transmission
  - Idle circuits – generate random data.

Parity Checking

- This error detection mechanism requires the sender to compute an additional bit, *the parity bit*, and to attach it to each character before sending.
- After all bits of a character arrive, the receiver removes the parity bit, and performs the same computation as the sender;
- It then verifies that the result agrees with the value of the parity bit.

Parity Checking

- If one of the bits in the character is damaged in transit, the receiver's computation will not agree with the parity bit and the receiver will report that an error occurred.

Forms of parity

- Even parity: To achieve even parity, the sender sets the parity bit to 0 or 1 (whichever will make the *total number of 1 bits an even number*).
- Examples: The parity bit of:
  - 0100101 is 1
  - 1010101 is 0

Forms of parity

- Odd parity: To achieve even parity, the sender sets the parity bit to 0 or 1 (whichever will make the *total number of 1 bits an odd number*).
- Examples: The parity bit of:
  - 0100111 is 1
  - 1010101 is 1
  - 1110000 is 0

Important Idea

- To detect errors, network systems usually send a small amount of additional information with data.
- A sender computes the value of the additional information from the data.
- A receiver performs the same computation to verify that the packet was transmitted without error.

Probability, Mathematics and Error Detection

- The parity mechanism detect a single-bit error – it cannot detect all possible errors.
- If the transmission errors change an even number of bits, the parity is preserved in all cases. Here's an example for 2 bits:
  - 00 to 11
  - 01 to 10
  - 10 to 01
  - 11 to 00

Detecting Errors with Checksums

H	e	l	l	o		w	o	r	l	d	.
48	65	6C	6C	6F	20	77	6F	72	6C	64	2E

4865 + 6C6C + 6F20 + 776F + 726C + 642E + carry = 71FC

Checksum: Implications

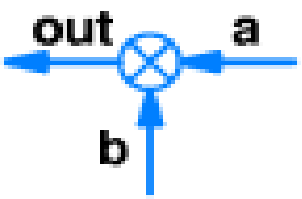
- Advantages
  - Easy of computation
  - Size
- Disadvantages
  - Does not detect all common errors

Detecting Errors with Checksums

Data Item In Binary	Checksum Value	Data Item In Binary	Checksum Value
0001	1	0011	3
0010	2	0000	0
0011	3	0001	1
0001	1	0011	3
totals	7		7

Illustration of how a checksum can fail to detect transmission errors. Reversing the value of the second bit in each data item produces the same checksum.

Detecting Errors with CRC

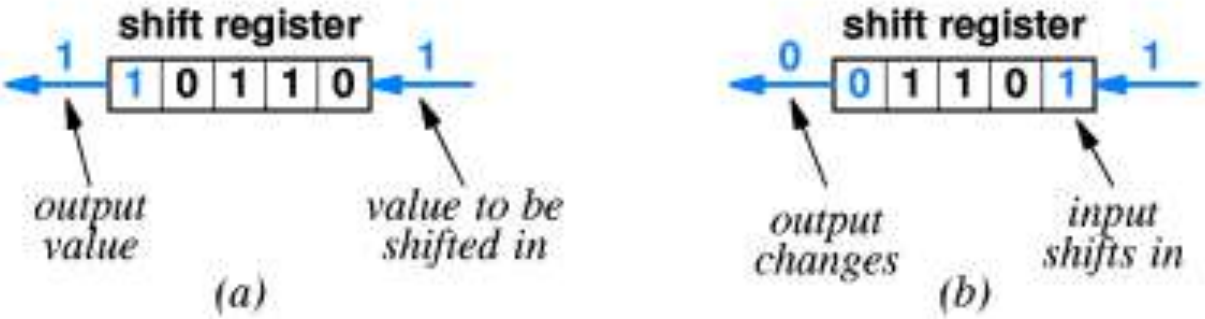


(a)

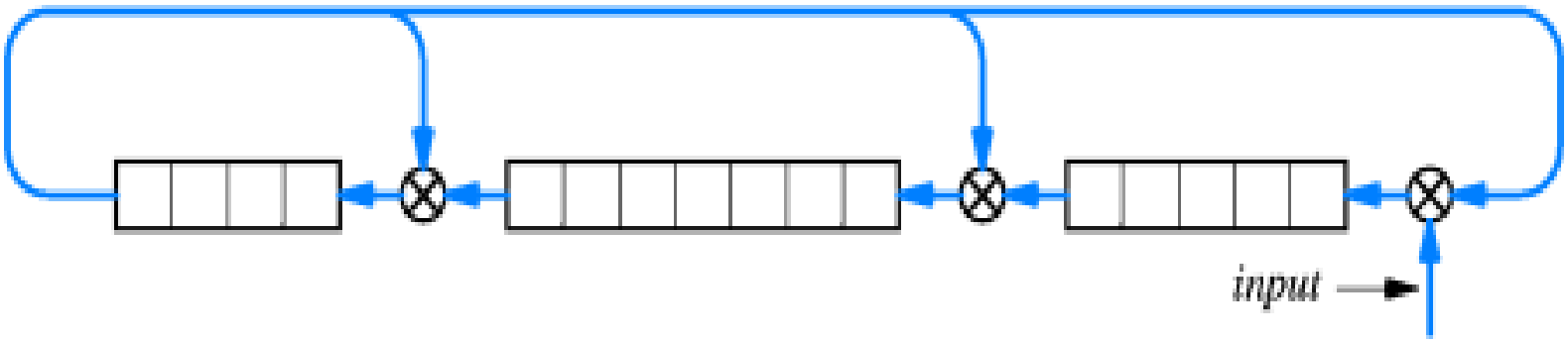
a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

(b)

Detecting Errors with CRC



Combining Building Blocks



A small modification to CRC

- Standard CRC algorithms use a small modification to the scheme described before.
- An additional 16 bits of zeroes are added to the message.
- Mathematically, these extra bits cause the resulting CRC to act as an inverse.
- The receiver computes the CRC over the incoming message plus the incoming CRC.
- If all bits are received correctly, the computed value will be zero.
- This technique is beneficial because HW can compare 16 bits to zero efficiently.

Examples of errors

- *Vertical errors*: A character oriented I/O device that is damaged might set the first two bits of every character to zero.
- *Burst errors:* Errors that involve changes to a small set of bits near a single location. Can be caused by:
  - Lightning
  - Electric Motors

Burst Errors

- It can be proved that CRC computation detects more errors than a checksum.
- CRC detects *vertical errors* better than checksums.
- CRC is especially useful for detecting *burst errors*.

Frame Format and Error Detection Mechanisms

Frame Format and Error Detection Mechanisms

- Is byte stuffing required for CRC?
- Because the CRC may have arbitrary values, one or both of the 8-bit values might correspond to a special character (soh, eot or esc).
- Two cases to consider:
  - The character can be damaged but not lost
  - If characters can be lost

Frame Format and Error Detection Mechanisms

- If characters can be damaged but not lost, a receiver can assume that the two characters immediately following EOT comprise a 16-bit CRC
- In this case byte stuffing of the CRC is not required.

Frame Format and Error Detection Mechanisms (cont)

- If characters can be lost, byte stuffing the CRC can add reliability
- Consider the case where the second character of the CRC is lost
- If the receiver assumes the two 8-bit characters following EOT form a CRC, the receiver will read the SOH from the next frame as part of the CRC.
- The receiver will skip the next frame (did not find SOH) and discard the current frame (CRC did not match).

Frame Format and Error Detection Mechanisms (cont)

- Should the CRC be computed over the original message or the stuffed frame?
- Individual standards specify whether the CRC is computed on the message or the encoded frame.

Summary

- To ensure prompt, fair service, networks divide data into small blocks called packets.
- A network multiplexes packets from multiple sources.
- To permit a frame to carry arbitrary data, the data must be modified before transmission and restored at reception.
- Data across a network can be mangled or lost.
- Networks include mechanisms to detect transmission errors.
- No error detection scheme is perfect.
- CRC is mathematically more complex, but can be implemented with simple and inexpensive hardware.